

Consultas avançadas com a ORM

orm/consultas.py e orm/analiticas.py — sete consultas sem SQL em string

Regra que valeu para todas

Nenhuma função destes dois módulos usa `text()` ou concatena SQL. As sete consultas são montadas com `select()`, `join()`, `where()`, `group_by()` e funções de agregação da biblioteca. Cada uma tem uma versão equivalente em SQL, no `10_etapa2_verificacao.sql` ou no `04_analiticas.sql`, e o resultado das duas tem de ser igual.

`aliased()` aparece em quase toda consulta

Paciente, residente e preceptor são todos pessoas. Uma consulta que precise do nome de dois deles junta a tabela `pessoa` duas vezes, e sem alias explícito o SQLAlchemy junta uma vez só, misturando as condições. O padrão adotado foi criar um alias por papel, com nome legível para facilitar a leitura do SQL gerado.

orm/consultas.py

```
pessoa_preceptor = aliased(Pessoa, name="pessoa_preceptor")
pessoa_paciente = aliased(Pessoa, name="pessoa_paciente")
```

As três consultas do item 5

Preceptores de pacientes flamenguistas

O enunciado pede os preceptores que supervisionaram residentes que atenderam pacientes com `is_flamengo` verdadeiro. A tabela `atendimento` guarda paciente, residente e preceptor na mesma linha, então a condição inteira cabe em uma linha de `atendimento` ligando os três. O JOIN com `Residente` não filtra nada por si, e foi mantido para deixar esse caminho explícito na consulta.

A consulta devolve também quantos atendimentos e quantos residentes distintos cada preceptor teve nessa situação. Com o `seed`, quatro dos cinco preceptores aparecem; Joao Nogueira fica fora, porque nenhum dos atendimentos dele envolve os pacientes flamenguistas, que são Ana Souza e Diego Ferreira.

Último atendimento de cada paciente

Uma função de janela resolve o "último de cada". `row_number` particionado por paciente e ordenado por data decrescente marca com 1 o atendimento mais recente, e o `desempate` por `id_cobre` dois atendimentos no mesmo instante. O resultado vira subconsulta e é reunido com a entidade `Atendimento`.

orm/consultas.py

```
ranking = (select(Atendimento.id_atendimento,
                  func.row_number().over(
                      partition_by=Atendimento.id_paciente,
                      order_by=[Atendimento.data_hora.desc(),
                                Atendimento.id_atendimento.desc()])
                  .label("posicao"))
           .subquery("ranking"))

stmt = (select(Atendimento)
       .join(ranking, ranking.c.id_atendimento == Atendimento.id_atendimento)
       .where(ranking.c.posicao == 1)
       .options(joinedload(Atendimento.paciente).joinedload(Paciente.pessoa),
                selectinload(Atendimento.procedimentos_realizados)
                .joinedload(ProcedimentoRealizado.procedimento),
                ...))
```

O carregamento adiantado entra por causa da resposta, que monta o nome do residente percorrendo `atendimento.residente.profissional.pessoa.nome`, três relacionamentos em cadeia. Sem as options, cada um deles viraria uma consulta por atendimento.

Com o seed, os cinco pacientes aparecem, todos com atendimento da segunda metade de julho de 2026, cada um com a lista de procedimentos daquele atendimento.

Percentual de procedimentos de alto risco por residente

A contagem condicional usa FILTER, que o PostgreSQL aceita em funções de agregação e que a DSL expõe como método do próprio agregado.

orm/consultas.py

```
total = func.count(ProcedimentoRealizado.id_procedimento)
altos = func.count(ProcedimentoRealizado.id_procedimento).filter(
    Procedimento.nivel_risco == "ALTO")

percentual = case(
    (total == 0, cast(0, Numeric(5, 2))),
    else_=func.round(cast(100.0 * altos / total, Numeric(10, 4)), 2))
```

A conta é por linha de procedimento realizado, não pela coluna quantidade. Duas aplicações de medicação no mesmo atendimento contam como um procedimento. Contar por quantidade daria outro número, também defensável, e a escolha está registrada no código.

As junções são externas para que "cada residente" inclua quem ainda não realizou procedimento nenhum. Nesse caso o total é zero, e o CASE evita a divisão por zero. A contagem usa uma coluna em vez de `count(*)`, porque com junção externa o `count(*)` devolveria 1 para o residente sem procedimentos.

Residente	Ano	Procedimentos	Alto risco	Percentual
Mariana Teles	R3	5	2	40,00%
Karina Duarte	R1	6	2	33,33%
Lucas Barbosa	R2	3	1	33,33%
Olivia Prado	R2	3	1	33,33%
Nathan Ribeiro	R1	3	0	0,00%

As quatro analíticas da Etapa 1

Estas rodavam pelo psql, a partir do 04_analiticas.sql. Foram reescritas em DSL porque o item 4 pede todas as operações da Etapa 1 reimplementadas com ORM, e ganharam endpoint em /orm/analiticas.

Ranking de residentes

outerjoin no lugar do LEFT JOIN. A junção externa mantém no resultado o residente que ainda não atendeu ninguém, com total zero, em vez de deixá-lo fora do ranking.

Preceptores acima de um total num mês

A consulta separa WHERE de HAVING. O filtro do mês vai no WHERE, antes de agrupar; a contagem mínima vai no HAVING, depois. Trocar um pelo outro daria resultado errado, porque o WHERE não enxerga o agregado e o HAVING deixaria atendimento de outro mês entrar na conta.

O mês e o mínimo são parâmetros do endpoint, em vez de valores fixos no texto da consulta como no SQL original. Com julho de 2026 e mínimo 5, o resultado é Fernando Alves com 6 atendimentos, que é o caso plantado no seed da Etapa 1. Os atendimentos acrescentados pelo seed da Etapa 2 usam outros preceptores, para que esse número continue valendo.

Plantões por residente em cada unidade

A tabela escala é uma grade semanal, sem data concreta, então "no mês corrente" admite duas leituras. As duas foram implementadas, e o endpoint escolhe pelo parâmetro projetar_no_mes. A primeira conta as posições fixas da grade. A segunda projeta a grade nos dias do mês atual, e é a mais interessante do ponto de vista da DSL.

orm/analiticas.py

```
inicio = func.date_trunc("month", func.current_date())
fim = inicio + literal_column("INTERVAL '1 month - 1 day'")
serie = func.generate_series(inicio, fim,
                             literal_column("INTERVAL '1 day'")).alias("dia")

nome_do_dia = case({0: "domingo", 1: "segunda", 2: "terca", 3: "quarta",
                    4: "quinta", 5: "sexta", 6: "sabado"},
                  value=cast(extract("dow", serie.column), Integer))

stmt = select(...).join(serie, Escala.dia_semana == nome_do_dia)
```

alias() é o que permite usar uma função geradora de linhas no FROM pela DSL. Sem isso, generate_series só apareceria dentro de um text(), e a consulta deixaria de cumprir a exigência do enunciado. Os valores do CASE seguem a constraint chk_dia_semana do esquema.

A primeira tentativa aqui usou table_valued("dia"), que parece a escolha natural pelo nome. Ela compila, mas o SQL sai com "AS anon_1" e depois referencia anon_1.dia, coluna que o PostgreSQL não reconhece: para função de coluna única, o banco usa o apelido da função como nome da coluna. O erro só apareceu na execução contra o banco, e não na compilação da consulta.

Pacientes sem procedimento de alto risco

A consulta usa exists() negado em vez de uma comparação com lista. Com NOT IN, um único NULL vindo da subconsulta faz a condição inteira deixar de casar, e o resultado vem vazio sem erro nenhum. NOT EXISTS não tem esse comportamento.

orm/analiticas.py

```
houve_alto_risco = (select(1)
    .select_from(Atendimento)
    .join(ProcedimentoRealizado, ...)
    .join(Procedimento, ...)
    .where(Atendimento.id_paciente == Paciente.id_pessoa,
           Procedimento.nivel_risco == "ALTO")
    .exists())

stmt = select(...).where(~houve_alto_risco)
```

A correlação com a consulta externa acontece na comparação `Atendimento.id_paciente == Paciente.id_pessoa`, que referencia a entidade de fora da subconsulta.

Como comparar as duas versões

A seção 9 do `10_etapa2_verificacao.sql` tem as três consultas do item 5 escritas em SQL, com os resultados esperados em comentário. O `04_analiticas.sql` tem as quatro da Etapa 1. Na interface, a aba Consultas ORM traz as sete, cada uma num cartão, com a explicação do critério adotado.